



CGgraph: An Ultra-fast Graph Processing System on Modern Commodity CPU-GPU Co-processor

Pengjie Cui^{1*}

Northeastern University
1810602@stu.neu.edu.cn

Haotian Liu²

Southern University of
Science and Technology
12231144@mail.sustech.edu.cn

Bo Tang^{2†}

Southern University of
Science and Technology
tangb3@sustech.edu.cn

Ye Yuan^{3†}

Beijing Institute of
Technology
yuan-ye@bit.edu.cn

ABSTRACT

In recent years, many CPU-GPU heterogeneous graph processing systems have been developed in both academic and industrial to facilitate large-scale graph processing in various applications, e.g., social networks and biological networks. However, the performance of existing systems can be significantly improved by addressing two prevailing challenges: GPU memory over-subscription and efficient CPU-GPU cooperative processing.

In this work, we propose CGgraph, an ultra-fast CPU-GPU graph processing system to address these challenges. In particular, CGgraph overcomes GPU-memory over-subscription by extracting a subgraph which only needs to be loaded into GPU memory once, but its vertices and edges can be used in multiple iterations during the graph processing procedure. To support efficient CPU-GPU co-processing, we design a CPU-GPU cooperative processing scheme, which balances the workloads between CPU and GPU by on-demand task allocation. To evaluate the efficiency of CGgraph, we conduct extensive experiments, comparing it with 7 state-of-the-art systems using 4 well-known graph algorithms on 6 real-world graphs. Our prototype system CGgraph outperforms all existing systems, delivering up to an order of magnitude improvement. Moreover, CGgraph on a modern commodity machine with a CPU-GPU co-processor yields superior (or at the very least, comparable) performance compared to existing systems on a high-end CPU-GPU server.

PVLDB Reference Format:

Pengjie Cui, Haotian Liu, Bo Tang, and Ye Yuan. CGgraph: An Ultra-fast Graph Processing System on Modern Commodity CPU-GPU Co-processor. PVLDB, 17(6): 1405 - 1417, 2024.
doi:10.14778/3648160.3648179

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/DBGROUP-SUSTech/CGgraph>.

* This work was done when Pengjie Cui was a research assistant at Southern University of Science and Technology.

† Both Bo Tang and Ye Yuan are corresponding authors.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 17, No. 6 ISSN 2150-8097.
doi:10.14778/3648160.3648179

1 INTRODUCTION

The scale of graphs often undergoes significant growth over time in various graph applications, e.g., social networks [13], web content [53], and biological networks [37]. The *vertex-centric computation model* [33] has been the *de facto* standard for large-scale graph processing due to its simplicity and scalability. For example, it has been widely adopted by various CPU-based distributed systems [21, 22, 28, 33, 36, 44, 56, 58]. Moreover, many GPU-based graph systems have been developed upon it [9, 20, 26, 31, 34, 39, 43, 48, 51, 55, 57] to further improve the performance of large-scale graph processing. We classify these GPU-based systems into two categories: (i) GPU-only computing systems, such as Cusha [26], Gunrock [51], Groute [9]; and (ii) CPU-GPU heterogeneous computing systems, including Totem [20], Subway [39], LargeGraph [55]. While GPU-only systems offer high performance, they fail to process large-scale graphs that exceed the GPU global memory capacity (a.k.a *GPU memory over-subscription*). To address it, CPU-GPU heterogeneous systems have been developed to mitigate the issue by leveraging both CPU and GPU resources.

However, there are still two open challenges, i.e., GPU memory over-subscription and efficient CPU-GPU Cooperative Processing, that need to be addressed in order to build an ultra-fast graph processing system on a modern commodity CPU-GPU co-processor. We elaborate the details of them in the following.

GPU memory over-subscription. Out-of-core graph processing has emerged as a prevalent approach to tackle GPU memory over-subscription in CPU-GPU heterogeneous systems. In particular, three prominent approaches have been explored in the literature.

(I) Partition-based approach [23, 27, 41]: The large graph is partitioned, and then “partitions” are loaded into the limited GPU global memory before each iteration during the graph processing procedure. While this approach has been improved by various techniques, such as asynchronously streaming data movement [23, 27, 41], and reducing data movement size by tracking active vertices or edges [23, 41], the benefits are still limited. The core reason is the data movement cost is expensive due to the limited bandwidth of the PCIe connection.

(II) Unified memory-based approach [19, 30]: With the advent of CUDA 8.0 and the Pascal architecture, GPU applications gained the ability to transparently access the CPU main memory. Thus, many graph processing systems load data pages from CPU main memory to GPU global memory by triggering page faults. This approach offers the advantage of on-demand data loading. However, the limitations of this approach are two-fold: (i) the page faulting process incurs overhead; and (ii) the on-demand loading of pages

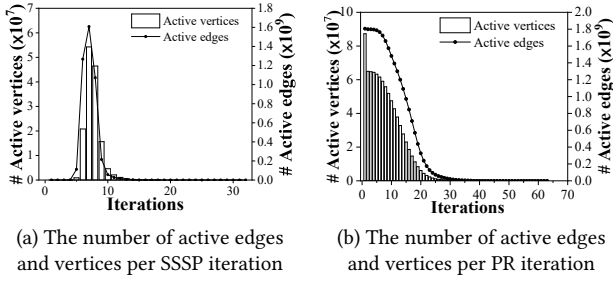


Figure 1: The statistics of running SSSP and PR on *friendster*

may include a large proportion of inactive edges/vertices, resulting in unnecessary data transfers.

(III) Subgraph generation-based approach [39]: Recently, a subgraph generation algorithm was proposed in Subway [39] to tackle GPU memory subscription, which generates a subgraph that only includes active edges and vertices in GPU memory. A suite of techniques (e.g., concise-and-efficient graph representation, GPU accelerated implementation) has been devised to optimize its generation cost. This approach improved the performance of large-scale graph processing by reducing the frequency and the size of loaded subgraph. Nevertheless, the generation cost of the subgraph in nearly every iteration cannot be ignored.

Efficient CPU-GPU Cooperative Processing. Existing CPU-GPU heterogeneous graph processing systems leverage both CPU and GPU to execute graph algorithms. However, the methods of cooperation between the CPU and GPU are significantly different among these systems. We next summarize the CPU-GPU cooperative methods in three groups.

(I) CPU and GPU are the first-class citizens [20]: It means both the CPU and GPU execute the computation tasks in each iteration during the graph processing procedure. For example, Totem [20] first loads the partition to GPU memory at the beginning, then the GPU cores will process the tasks of the active edges and vertices in its memory, and the rest active edges and vertices will be executed by CPU cores. The CPU and GPU are executing different computation tasks in each iteration. Obviously, the workload distribution between the CPU and GPU can vary significantly across iterations.

(II) GPU is the first-class citizen [23, 39, 41]: The GPU is primarily responsible for executing the computational tasks in each iteration in this method, and the CPU plays a supportive role and assists the GPU in various ways. For instance, the CPU identifies the active vertices that serve as input for the GPU-accelerated subgraph generation algorithm in Subway [39]. There is no doubt that the utilization of the CPU could be further improved, as it may remain idle while the GPU is heavily occupied with task processing.

(III) Hybrid roles of CPU [55]: This method combines the utilization of the CPU in both (I) and (II). The CPU initially acts as an assistant to the GPU, as described in (II). However, it also executes partial computation tasks during each iteration of graph processing. For example, the CPU first identifies the frequent paths in the graph and assigns the tasks which are in frequent paths to GPU in LargeGraph[55]. After that, it processes the computations task in infrequent path. Nevertheless, the method encounters task-level imbalances during graph processing. The reason is that the number

Table 1: The range of speedup times of CGgraph

Dataset	BFS	SSSP	WCC	PR
<i>gsh2015tpd</i>	0.74-2.35X	1.22-4.44X	0.97-4.72X	1.25-2.23X
<i>twitter2010</i>	1.53-3.39X	2.18-4.68X	2.01-5.11X	2.70-9.73X
<i>friendster</i>	2.05-3.48X	4.45- 11.94X	3.42- 11.07X	2.66-7.83X
<i>weibo</i>	2.11-2.94X	2.43-6.05X	2.81-7.50X	2.39-6.35X
<i>uk-2006</i>	0.96- 4.15X	2.46-5.37X	1.58-3.91X	2.33-6.96X
<i>un-union</i>	1.30-2.86X	2.05-2.61X	1.45-2.82X	1.79-2.79X

of active edges for a vertex in the frequent path is obvious larger than it in the infrequent path.

In this work, we propose CGgraph, an ultra-fast graph processing system on a CPU-GPU co-processor, to simultaneously address the above two open challenges. Moreover, its performance improvement does not rely on the high-end CPU-GPU servers, as it is built on a modern commodity machine with a CPU-GPU co-processor.

With the above discussion, the key limitation of all existing solutions for GPU memory over-subscription is that the benefits obtained by exploiting the high parallelism of GPU are neglected due to the data movement overhead. In CGgraph, we address this by following the “load once and use multiple times” principle. Specifically, Figure 1(a) depicts the number of active edges and vertices in each iteration by running Single Source Shortest Path (SSSP) and PageRank (PR) algorithms on *friendster*, which includes 124 million vertices and 1,806 million edges. Obviously, the number of active edges and vertices undergoes significant variation across iterations in both SSSP and PR algorithms. Hence, the general idea to address GPU memory over-subscription in CGgraph is extracting a size-constrained subgraph G' from graph G , in which its vertices and edges can be active in multiple iterations. Thus, CGgraph loads it to GPU global memory before graph processing and keeps it in GPU during the whole graph processing procedure.

For the CPU-GPU cooperative processing, the first pitfall of existing solutions is that the GPU should be used in every iteration during graph processing. The overhead to invoke GPU will definitely be larger than its benefits in those iterations which only has a few active edges and vertices, e.g., 1st-3rd iterations and 11th to 31st iterations in SSSP and 37th to 63th iterations in PR, as shown in Figures 1 (a) and (b), respectively. In CGgraph, we devise a simple-yet-effective GPU invoking strategy to decide whether the GPU will be invoked or not in each iteration. Moreover, the most important limitation of existing solutions is that almost all of them (if not all) lack of fine-grained (i.e., vertex-level and edge-level) CPU and GPU workload balance scheme. In CGgraph, we devise (i) an edge-level CPU-GPU cooperative computation scheme and (ii) on-demand task allocation approach to overcome it. To further unlock the computation capabilities of CPU and GPU, a suite of optimizations (e.g., task stealing, inter-warp balancing strategy) for task processing on CPU and GPU have been devised in CGgraph.

We conduct extensive experiments to demonstrate the superiority of our CGgraph. Specifically, we report the minimum and maximum speedup of CGgraph over 7 state-of-the-art systems by running 4 widely used graph algorithms, i.e., Breadth-First-Search (BFS), Weakly Connected Components (WCC), SSSP and PR on 6 public graph datasets in Table 1. CGgraph achieves up to 4.15X, 11.94X, 11.07X and 9.73X speedup over existing systems by processing BFS, SSSP, WCC and PR, respectively. Interestingly, the

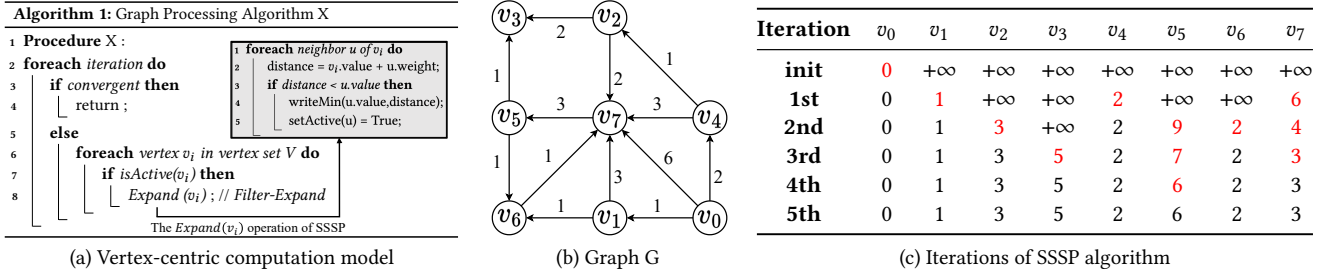


Figure 2: Graph computation model and its illustrative example of SSSP algorithm

performance of our proposed CGgraph on modern commodity machine is even better than (or at least comparable with) the performance of existing systems on a high-end CPU-GPU server. This remarkable achievement confirms the efficiency and capability of CGgraph in leveraging CPU and GPU hardware resources.

The rest of the paper is organized as follows. In Section 2, we introduce the graph processing computation model and its research challenges. In Section 3, we overview the architecture of CGgraph. The techniques to address the above two open challenges are introduced in Sections 4 and 5, respectively. The experimental evaluation results are presented in Section 6. We conclude the paper and highlight the future research directions in Section 7.

2 COMPUTATION MODEL AND ITS CHALLENGES

In this section, we first introduce the widely-used computation model in graph processing systems, then we highlight the research challenges to build a fast graph processing system on a modern commodity machine with a CPU-GPU co-processor.

2.1 Graph Processing Computation Model

Many graph processing systems employ the vertex-centric computation model (a.k.a., programming model) due to its high scalability [21, 51, 58]. As depicted in Figure 2(a), the vertex-centric computation model adopts an “iterative-convergent” process. In particular, the given graph algorithm is processed in an iterative manner, terminating when all vertex values do not change or when the procedure reaches a limited iteration time, known as “convergent” in Line 3, Algorithm 1. For each iteration, every vertex in the graph is processed by *Filter-Expand* operation [34, 35]. We refer to the *Filter-Expand* operation of a vertex as a computation *task* in the graph processing system. In particular, the *Filter* step selects the active vertices to execute the *Expand* (see Line 7), and the inactive vertices are filtered. The *Expand* step processes the input active vertices, and generates new active vertices for the next iteration. More specifically, the *Expand* step maintains algorithm-specific logic for vertex processing, ensuring that all active vertices are processed with the same algorithmic logic. The inside box in Figure 2(a) shows an example of the *Expand* step for SSSP algorithm, which computes the shortest distance from a given source vertex to all other vertices on a weighted directed graph. The vertex-centric computation model is generic as it can be instantiated to various graph processing algorithms, e.g., BFS, PR, WCC, by only changing the algorithmic logic in the above *Expand* step. Moreover, the computation tasks of

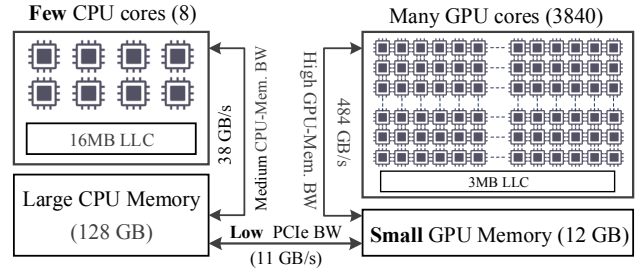


Figure 3: The architecture of CPU-GPU co-processor

all vertices in each iteration can be parallel processed inherently via high-parallelism hardware (e.g., multi-threads CPU or GPU).

Example. Taking the graph G in Figure 2(b) as an example, we illustrate the vertex-centric computation processing procedure of SSSP with source node v_0 in Figure 2(c). At the beginning of the iteration, the vertex values, i.e., the found shortest distance from the source vertex v_0 so far, are initialized to $+\infty$. For *Expand* step of v_0 , it updates its vertex value to 0 and activates its neighbors $\{v_1, v_4, v_7\}$, which will be processed in the next iteration. In the 1st iteration, the vertex values of v_1, v_4 and v_7 are computed by their corresponding *Expand* step, see the red values in the 1st iteration row. The newly active vertex set is $\{v_2, v_5, v_6, v_7\}$, and it will be processed in the 2nd iteration. As illustrated in Figure 2(c), the processing procedure terminates after the 5th iteration, as all the shortest distances from source v_0 are computed.

In the literature, the subgraph-centric computation model is also used in various graph processing systems [38, 45, 46]. We omit it in this work as it is not trivial to accelerate it by CPU-GPU co-processing [24], e.g., how to exploit single instruction multiple threads architecture of GPU to parallel process different subgraphs?

2.2 Research Challenges

In this section, we first introduce the characteristics of CPU-GPU co-processors in modern commodity machine, then highlight the challenges to build an ultra-fast graph processing system upon it.

CPU-GPU Co-processor. Figure 3 shows the architecture of a CPU-GPU co-processor in a modern commodity machine, i.e., DELL OptiPlex 7000MT Desktop Computer. There are several important properties in a CPU-GPU co-processor. First, the GPU has a larger number of cores (i.e., 3840 cores) compared to the CPU (with 8 cores), enabling high parallelism computation in the GPU. Second, GPU utilizes high-bandwidth memory (HBM), which is higher (484 GB/s) than the memory access bandwidth of the CPU (38 GB/s).

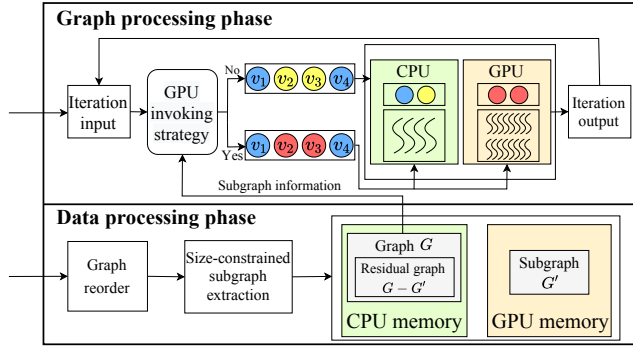


Figure 4: CGgraph architecture overview

Last but not the most important, the size of the CPU main memory is always significantly larger than the size of the GPU global memory, e.g., 128GB vs. 12GB. The data movement from CPU memory to GPU memory is very expensive, as the bandwidth of PCIe is limited, e.g., 11 GB/s, which is the core challenge to design an out-of-GPU-memory data processing system [23, 32, 39, 41, 50, 55].

Research Challenges. The performance of many CPU-GPU heterogeneous graph processing systems could be further improved as: (i) the gained parallel computing benefits from GPU are far smaller than the extra data movement cost, and (ii) existing systems do not fully utilize the computation power of CPU and GPU. Hence, there are two research challenges to build an ultra-fast graph processing system on modern commodity CPU-GPU co-processor:

C1: GPU memory utilization is quite low. As elaborated in Section 1, existing systems utilize GPU global memory by (i) loading data partitions into it at each iteration explicitly [23, 41, 55], and (ii) generating a subgraph which only includes active edges and vertices at nearly every iteration [39]. However, both methods incur significant overheads, e.g., the data movement cost or subgraph generation cost. Thus, the first challenge of our work is improving the utilization of the limited GPU global memory.

C2: CPU-GPU cooperation is inefficient. Specifically, existing systems either use CPU memory as the secondary storage [23, 39, 41] or assign computation tasks to CPU via a fixed strategy [20, 55]. The limitations are two-fold: (i) the co-processing between CPU and GPU is implicit, which results in workload imbalance, and (ii) they do not fully unlock the computation capabilities of the CPU and GPU. Hence, the second challenge of our work is cooperating CPU and GPU effectively.

3 THE OVERVIEW OF CGGRAPH

Figure 4 depicts the architecture of our proposed CGgraph. It consists of two phases: (i) data processing phase, and (ii) graph processing phase, which are designed to address the two challenges in Section 2, respectively. In particular, it first applies the graph reordering algorithm on the raw graph data. Then, CGgraph extracts a size-constrained subgraph G' from the reordered graph G , in which its vertices and edges can be active in multiple iterations, and loads it into GPU global memory. We will elaborate on the details of data processing techniques in Section 4. For the graph processing phase, CGgraph employs the widely used vertex-centric computation model as other studies (see Section 2) to process graph

algorithms. For each iteration, the GPU invoking strategy first decides how to process the active vertices/edges, i.e., CPU-only or CPU-GPU co-processing by exploiting the statistics of the subgraph G' . Then, all computation tasks will be executed accordingly. The details of the proposed techniques in the graph processing phase are described in Section 5.

4 DATA PROCESSING PHASE IN CGGRAPH

As the data movement cost between CPU main memory and GPU global memory is very expensive, the principle of our solution is: “load once and use multiple times!” With the above idea in mind, our research goal is extracting a subgraph G' from the input large graph G , such that (i) only load it to GPU global memory once, and (ii) it unlocks the high parallelism computation capability of GPU among multiple iterations. In this section, we first highlight our observed insights in the graph processing computation model (Section 4.1), which guides us to design the two core subroutines in CGgraph: graph reordering and size-constrained subgraph extraction algorithm, as presented in Sections 4.2 and 4.3, respectively.

4.1 Observed Insights

Returning back the SSSP example (in Figure 2) of the graph processing computation model in Section 2, we found vertex values of v_5 and v_7 are updated in three iterations during the graph processing procedure. Specifically, the vertex value of v_7 is updated by $v_0 \rightarrow v_7$, $v_0 \rightarrow v_1 \rightarrow v_7$, and $v_0 \rightarrow v_1 \rightarrow v_6 \rightarrow v_7$ in the 1st, 2nd, and 3rd iteration, respectively. Moreover, the vertex value v_5 also be updated in the 2nd to 4th iterations as it is an out-going neighbor of v_7 . We then summarize the first insight according to the above observation.

Insight 1. The numbers of active times of different vertices in the graph are quite different. Vertices with a large number of in-coming edges are likely to be active in many iterations during the graph processing. Consequently, the out-going neighbor of such vertices will also be affected in many iterations.

Another interesting fact is that v_3 is a special type of vertices (a.k.a sink vertices), which do not have any out-going neighbors, thus, the vertex value of v_3 does not influence any other vertex value in the whole processing procedure. Moreover, the vertex value of v_3 depends on the vertex values of its in-coming neighbors v_2 and v_5 . Hence, the second insight we observed is as follows.

Insight 2. The sink vertices do not influence other vertices. Thus, their vertex values will not affect the values of other vertices. In particular, the vertex value of a sink vertex can be computed after all its in-coming neighbors’ vertex values are finalized, for example, computing them in the last iteration.

4.2 Graph Reordering

With the above observed insights, a general idea is to selectively load the vertices that have a high number of in-coming neighbors into the GPU’s constrained memory, while excluding the sink vertices. This problem is a variant of the well-known dense subgraph extraction problem [29]: *given a sparse graph, how to find a meaningful dense subgraph?* One of the commonly used methods is reordering and positioning the vertices or edges to a specific

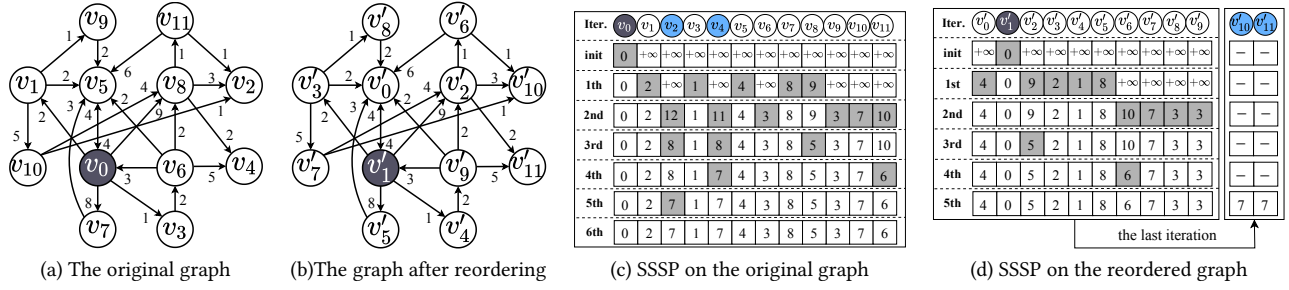


Figure 5: The effect of graph reordering on SSSP algorithm

location, then extracting a size-constrained subgraph upon the re-ordered graph. For example, Chen et. al. [11] reorders the adjacency matrix and gathers the dense part into the diagonal, then proposes a blocking algorithm to find the dense diagonal block. Inspired by it, we devise a graph reordering algorithm which stores the vertices with more in-coming neighbors at the front while placing the sink vertices at the end. In addition, we observed that when a vertex finishes its computation, its neighbors are likely to be active in the next iteration during the graph processing. We explicitly take this observation into consideration to enhance data access locality. Therefore, the reordering algorithm ensures that the out-going neighbors of a vertex with a large number of in-coming neighbors are situated nearby.

To achieve the above goals, we propose a heuristic graph reordering algorithm as follows, which takes all vertices V of the graph as input, and outputs the reordered list S'_V . It sorts all vertices in V by the descending order of its in-degree at first, denoted by S_V . Then we slightly revise the Breadth-First-Search (BFS) algorithm to traverse all these vertices in the graph to obtain the reordered list S'_V . Specifically, the revised BFS algorithm starts with the first non-visited vertex in the sorted vertex array S_V . During the traversal procedure, each visiting vertex is either appended to the reordered list S'_V (i.e., non-sink vertex) or appended to the sink vertices list S_{sink} (i.e., sink vertex). The above traversal steps repeat until all vertices have been visited. Last, the sink vertices list S_{sink} is appended to the reordered list S'_V . We refer the interested reader to Appendix A in technical report [2] for the pseudocode and the time complexity analysis of the graph reordering algorithm.

The benefits of our proposed graph reordering algorithm are three-fold: (i) it reorders the vertices and edges which will be active frequently during graph processing. It improves the utilization of GPU global memory as we will elaborate in Section 4.3; (ii) it explicitly moves the sink vertices of the graph to the end of S'_V , which significantly reduces the computation cost of graph processing algorithms; and (iii) it increases CPU and GPU cache hit rates as it takes the data access locality into consideration. We next demonstrate the effectiveness of the above graph reordering algorithm via the example in Figure 5.

Example. Figure 5(a) shows the original graph, and its reordered graph is presented in Figure 5(b). The subscripts of v and v' represent the order of a vertex in the original graph and reordered graph, respectively. The graph reordering algorithm changes the id of each vertex, e.g., v_0 in the original graph is the same as v'_1 in the reordered graph. We run the SSSP algorithm on both graphs. Their corresponding iterations are shown in Figures 5(c) and (d),

respectively. In particular, the sink vertices in the reordered graph are located at the end, which are marked as blue. We update them only after the vertex values of other vertices have been finalized. Obviously, moving the sink vertices to the end and postponing their vertex value computation via the reordered graph significantly reduces the computation cost when we compare the iterations in the original graph and reordered graph. For example, the vertex values of sink vertices v_2 and v_4 are updated three times, see the columns with blue color vertex in Figure 5(c). However, they are only updated once in the reordered graph, as the last two columns in Figure 5(d).

Moreover, the graph reordering algorithm explicitly improves data access locality in each iteration. In both Figures 5(c) and (d), the gray cells in each row are the accessed vertices in each iteration. Visually, the data access locality on the reordered graph is preserved much better than on the original graph, as the visited vertices of each iteration in Figure 5(d) are located in continuous grids. Taking the 2nd iteration as an example, the original graph visits $v_2, v_4, v_6, v_9, v_{10}$, and v_{11} . However, four of these vertices are consecutively positioned from v'_6 to v'_9 in the reordered graph, and two are sink vertices that do not need access at this iteration.

Relevant studies. Many graph reordering algorithms [10, 14, 15, 17, 52, 54] have been studied in the literature for various optimization goals. For example, RCM [14] rearranges the matrix representation of a graph into a band matrix with a narrow bandwidth. Gorder [52] enhances the average neighborhood overlap of connections between adjacent nodes by maximizing the shared edges within blocks of consecutive nodes (sliding window) with size w to reduce the cache misses. Adapting them to our problem is possible, but the overall performance of CGgraph will be worse than our above graph reordering algorithm. The primary reason is that our goal is to reorder the vertices based on their in-degree while explicitly considering data access locality, and we will verify this in Section 6.

Generality discussion. The above graph reordering algorithm is generic, and it can be used to improve the performance of all existing graph processing systems (e.g., Totem [20], Subway [39]) as it enhances the data access locality in an explicit way. We will confirm it in Section 6.

4.3 Size-constrained Subgraph Extraction

Before introducing our size-constrained subgraph extraction algorithm, we demonstrate the effectiveness of our graph reordering algorithm, which moves the vertices with high in-coming neighbors

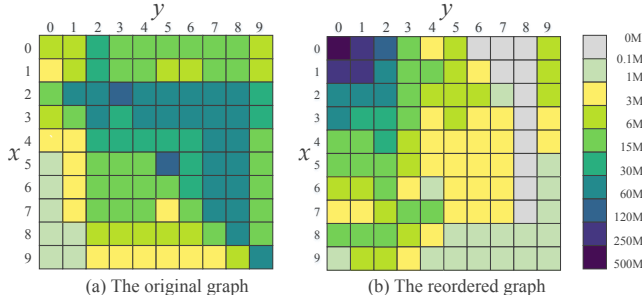


Figure 6: The heatmaps of the edge distribution

to the front. Figures 6(a) and (b) show the heatmap of edge distributions among the vertex group pairs of *twitter2010* in the original and reordered graph, respectively. The deeper (resp. lighter) the color of a cell, the larger (resp. smaller) the edges in this vertex group pair. Specifically, *twitter2010* has 42 millions of vertices, which are equally divided into 10 groups. For example, the first 4.2 millions vertex are assigned to group 0. An edge (v_i, v_j) in the graph G is counted in vertex group pair (x, y) if and only if its start vertex v_i is in the vertex group x and its end vertex v_j is in the vertex group y . Comparing with the heatmap of the original graph in Figure 6(a), the deeper color cells are concentrated on the left-upper corner of the reordered graph heatmap in Figure 6(b). The reason is that our graph reordering algorithm moves the vertices with high in-coming neighbors to the front. Thus, the group pairs at the left-upper corner are probably include more edges than others. In addition, it is not surprising that the color of the cells in right-bottom corner is very light as these are sink vertices.

With the reordered graph, we next present our size-constrained subgraph extraction method as follows. First, it extracts a subset of vertices from the beginning of the ordered vertex list. Second, we identify the edges whose starting vertex and ending vertex are in the set of extracted vertices. The extracted vertices and corresponding selected edges form the subgraph G' , which will be loaded into GPU memory to support efficient graph processing. The exact number of the vertices to be extracted in the first step is determined as follows. We first accumulate the total out-going neighbors of the vertices from the ordered list. We then binary search the foremost vertex of which the total size of all of their out-neighbors can be fit into the size-constrained GPU memory.

The details of subgraph extraction algorithm. Different subgraphs of the same original graph will be extracted for various graph algorithms. For each graph algorithm on a specific graph, we determine the constrained memory size by excluding the additional GPU memory overhead of the graph algorithm from the GPU global memory budget. The additional GPU memory overhead of each graph algorithm is easy to estimate as it is only used to synchronize the intermediate results among CPU and GPU, we will elaborate shortly in Section 5. However, all these extracted subgraphs share the same starting vertex, i.e., the vertex at the beginning of the reordered list S'_v , as the purpose of the subgraph extraction algorithm is identifying those vertices are probably to be activated in multiple iterations during graph processing.

5 GRAPH PROCESSING PHASE IN CGGRAPH

In this section, we first propose a CPU-GPU cooperative computation scheme for fast graph processing on a single machine with CPU-GPU co-processor in Section 5.1. Next, we devise the processing optimization techniques on CPU and GPU, which fully unlock their computation capabilities, in Sections 5.2 and 5.3, respectively. Last, we present a simple-yet-effective strategy to decide whether CGgraph invokes GPU or not at each iteration in Section 5.4.

5.1 CPU and GPU Cooperative Processing

After the data processing phase in Section 4, the graph G is reordered and the subgraph G' which includes those selected high in-degree vertices (and their corresponding selected edges) is loaded into the GPU global memory. It is not trivial to parallel execute the graph processing algorithms via the vertex-centric computation model under the above setting. The major reason is that the subgraph G' , which is stored in GPU memory, probably only includes a subset of edges for every vertex in it.

In this section, we propose a CPU-GPU cooperative computation scheme to address it. For each iteration, every vertex in graph G is classified into two categories: (i) CPU-only and (ii) CPU-GPU. In particular, if the vertex v is not in subgraph G' , i.e., it is not in the GPU memory, it only can be processed on the CPU, thus, vertex v is in the CPU-only category. Otherwise, vertex v is in the CPU-GPU category. It is easy to process the CPU-only vertices via the vertex-centric computation model. Next, we present our CPU-GPU cooperative processing scheme for the CPU-GPU category vertices. For each CPU-GPU vertex v , two computation tasks will be instantiated. One of its computation tasks will process a subset of its edges (which is included in G') on GPU, the other computation task of it will process the rest of its edges in the residual graph, i.e., $G - G'$ on CPU. After that, the computed results of both tasks will be aggregated to derive the final result of vertex v .

For implementation-wise, the last aggregation step of every CPU-GPU vertex will do in a batch at the end of each iteration. In particular, every vertex value of GPU-processed computation tasks will be moved back to the CPU main memory together. Each of them will be aggregated with its corresponding vertex value on the CPU main memory to derive the final result. In addition, the updated value of every node in subgraph G' will be synchronized to the GPU memory at the beginning of the next iteration to guarantee the GPU computation tasks will not work on the out-of-date values. With the above vertex value synchronization method, the CPU and GPU computation task of the same vertex does not need to be processed at the same iteration. The value movement overhead is quite small as our subgraph extraction approach in Section 4.3 only extracts a subset of vertices from G , and it is the additional GPU memory overhead of the corresponding graph algorithm. We will verify it by experiments in Section 6.

Example. We use the graph G in Figure 7 as an example to illustrate how to cooperate CPU and GPU to process the SSSP algorithm via the vertex-centric computation model. Figure 7(a) is the reordered graph G , the gray part in it is the subgraph G' on GPU memory, see Figure 7(b), which is selected by the data processing phase. The black part is the residual graph, i.e., $G - G'$. To improve the computation efficiency, for the CPU-GPU co-processing vertex v ,

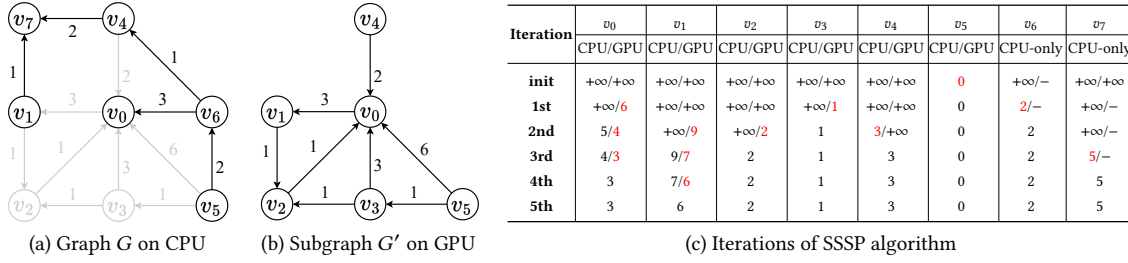


Figure 7: Cooperative CPU-GPU computation and its illustrative example of SSSP algorithm

CGgraph only processes its edges on the residual graph $G - G'$ via its computation task on the CPU.

Given the source node v_5 , Figure 7 (c) shows the SSSP algorithm processing procedure. The computation tasks of v_6 and v_7 are CPU-only and the rest vertices in G are in the CPU-GPU category, as the first row in Figure 7(c) shown. The distance of the source vertex is initialized to 0 and others are set to $+\infty$ in CPU memory at first. During the first iteration, the initialized vertex values are copied to GPU memory before computation task execution. The vertex value, i.e., the shortest distance of it found so far, of vertex v_6 , is updated by CPU-only computation task as v_6 is not in the subgraph G' . The vertex value of v_0 and v_3 are updated by their corresponding GPU-based computation task, and the values are moved to the CPU main memory for final aggregation. For example, the distance value of v_0 and v_3 on the CPU main memory are updated to 6 and 1, respectively, as the red color highlighted values at the 1st iteration row in Figure 7(c). Moreover, the vertices v_0 , v_1 , v_2 , and v_4 are active and waiting for processing at the next iteration. For the second iteration, the updated vertex values of v_0 to v_5 are moved to GPU global memory at first. The distance values of v_1 and v_2 are updated by their corresponding GPU computation tasks. However, the distance value of v_4 is updated by its CPU computation task as there is an edge (v_5, v_4) in the residual graph. For CPU-GPU vertex v_0 , its CPU computation task updates its value to 5 via the path $v_5 \rightarrow v_6 \rightarrow v_0$, and its GPU computation task updates its value to 4 via the path $v_5 \rightarrow v_3 \rightarrow v_0$. The final distance value of v_0 is obtained by aggregating the results of CPU and GPU computation tasks, i.e., $\min\{5, 4\} = 4$, as red color highlighted. CGgraph executes the SSSP algorithm following the above steps in each iteration until all the distance values do not change anymore.

On-demand task allocation. Even though the above CPU and GPU cooperative processing scheme is very efficient, it still has room for further performance improvement. Specifically, the vertices in the CPU-GPU category (i.e., in subgraph G') will be co-processed via CPU and GPU by default. However, it may incur the workload imbalance between the CPU and GPU if there are too many vertices waiting for execution on the GPU. In this section, we alleviate the workload imbalance issue by proposing the following on-demand task allocation approach.

Figure 8 shows our on-demand task allocation approach for CPU-GPU co-processing. At the beginning of each iteration, the vertices will be classified into CPU-only (in blue color) and CPU-GPU (in red color) categories, as the task box in Figure 8 shown. We then invoke CPU and GPU to cooperatively process these tasks by the

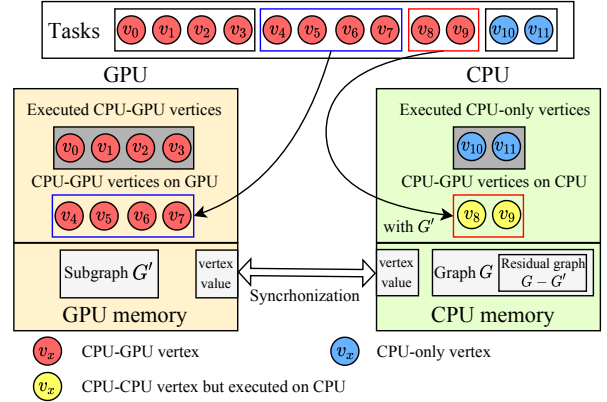


Figure 8: On-demand task allocation

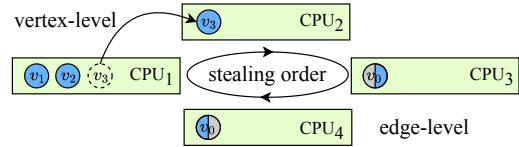


Figure 9: Task processing on CPU

above CPU-GPU cooperative processing scheme. When the CPU executed all these CPU-only vertices, it will start to process the CPU-GPU vertices, which originally are waiting for GPU execution, as the tasks of v_8, v_9 in the red box of Figure 8 shown. Specifically, the CPU-GPU vertices will not be executed by the GPU anymore if they are allocated to the CPU for execution as all their edges can be obtained from the graph G , which is in the CPU's main memory. As illustrated in Figure 8, the computation task of v_8 and v_9 (which originally are CPU-GPU vertices in red color) are executed by the CPU with all their edges in G , see the yellow vertices in Figure 8.

Until now, we devise a CPU-GPU cooperative processing scheme with on-demand task allocation approach to cooperate with the CPU and GPU for graph processing in CGgraph. In subsequent, we present the performance optimization techniques for efficient task processing on CPU and GPU in Sections 5.2 and 5.3, respectively.

5.2 Tasks Processing on CPU

To improve the utilization of every CPU core, we devise vertex-level and edge-level work stealing mechanisms in this section.

Vertex-level work stealing. For every vertex in each iteration, their corresponding computation tasks are uniformly assigned to

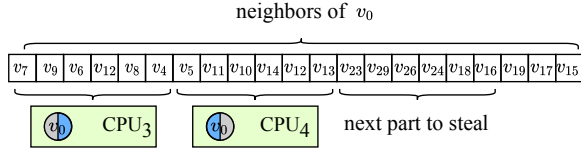


Figure 10: Edge-level work stealing

the task queue of each CPU core, and wait for execution. As shown in Figure 9, each CPU core has its own working queue, which includes the assigned tasks in this iteration. Each CPU core executes them one by one from the head to the tail of its working queue. The computation intensiveness of different tasks is quite different, which results in the different completion time for each CPU core. We devise vertex-level work stealing method to address the above load imbalance issue among CPU cores.

Take Figure 9 as an example, suppose there are three computation tasks in CPU₁ (i.e., v_1 , v_2 and v_3) when CPU₂ completes. Then, CPU₂ will steal tasks from the tail of CPU₁'s working queue, i.e., the computation task of v_3 . We impose a clockwise stealing order to guarantee the correctness and efficiency of execution, e.g., CPU₂ will steal the tasks in the working queue of CPU₁ first, it will steal the task of CPU₄ if and only if it cannot steal tasks from CPU₁. In our implementation, the CPU core will steal 64 tasks at a time as we use a bitset with 64 bits to filter the inactive vertices.

Edge-level work stealing. The execution time of computation tasks in each iteration always follows the long-tail distribution, e.g., a few vertices incur a very long execution time. To alleviate the load imbalance issue caused by a few long computation tasks, we further provide a fine-grained work stealing method on the CPU, i.e., edge-level work stealing. For example, the CPU₄ steals partial work of task with vertex v_0 , which is executing by CPU₃, as shown in Figure 9. We adapt the work splitting strategy [16] to split a vertex that has a large number of neighbors. In particular, when the CPU core processes a task associated with a large number of edges, i.e., it exceeds a threshold ϵ , we will mark it as “splittable”. It means that the task can be stolen partially by other CPU cores, we use the same stealing order as the vertex-level stealing method. For a “splittable” task, each CPU core only processes a subset of edges when processing it, and the rest edges can be processed by other CPU cores. For example, as shown in Figure 10, the CPU₃ processes the first 8 edges when it executes the task with v_0 , and the CPU₄ steals the task v_0 with the second 8 edges and executes it. Therefore, multiple CPU cores work together to complete the computation task of v_0 .

In summary, for all the computation tasks on the CPU, we first employ the vertex-level work stealing method to balance the workload among all CPU cores. After that, the edge-level work stealing method is applied to reduce the execution time of the tasks which have a large number of neighbors.

5.3 Tasks Processing on GPU

We exploit the computation ability of GPU by reducing the warp divergence during computation task processing on it. In the literature, cooperative thread array (CTA) [9, 48, 51] is a widely-used method to execute the computation tasks on GPU. It processes 256

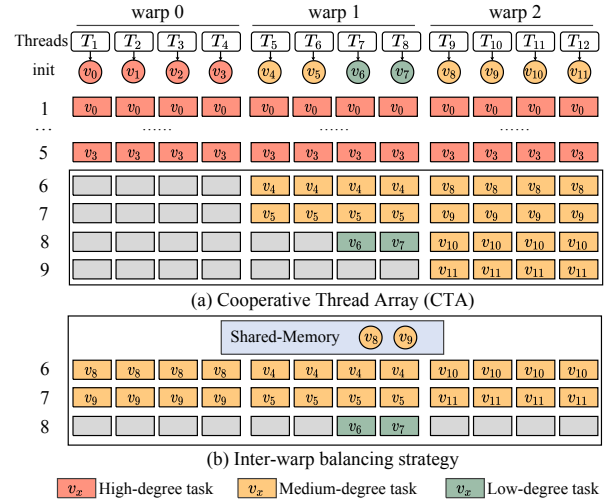


Figure 11: Task processing on GPU: CTA vs. IWB

computation tasks in a batch. CTA executes the high-degree tasks (i.e., the corresponding vertices have more than 256 edges) in a block manner, i.e., all 256 threads in a block will execute the task together. The medium-degree tasks, whose edges are in the range of 32 to 256, will be executed in a warp manner, i.e., use 32 threads in a warp to execute it. The low-degree tasks (i.e., less than 32 edges of their vertices) will be executed by a single thread.

Figure 11(a) depicts the task processing procedure example of CTA for a batch of computation tasks. It executes high-degree tasks (i.e., tasks of v_0 , v_1 , v_2 , v_3) by employing all threads across all warps in the block. The medium-degree tasks will be executed by all threads within a warp. For example, tasks of v_4 , v_5 and tasks of v_8 , v_9 , v_{10} , v_{11} are processed by the threads in warp₁ and warp₂, respectively. The low-degree tasks are executed by a single thread, see tasks of v_6 and v_7 . CTA achieves high GPU utilization via different execution manners for different tasks. However, it incurs significant inter-warp divergence when processing medium-degree tasks. For example, the threads in warp 0 are stalled after they processed the task with v_3 .

To address the inter-warp divergence on GPU, we re-assign the medium-degree tasks from overloaded warps to others. In particular, we calculate the average medium degree among warps. If the medium-degree tasks in a warp are larger than the average, we move the over part to the shared memory. For example, the average number of medium-degree tasks is 2 (6 tasks and 3 warps). Thus, the overloaded warp 2 will move two tasks (i.e., tasks of v_8 and v_9) to shared memory. Thus, both tasks can be executed by the threads in warp 0 after the processing of the task with v_3 , see Figure 11(b). In the implementation, we guarantee the number of tasks in the shared memory will not exceed the size of it. In addition, we apply the warp appending method [48] to visit the shared memory to reduce the number of atomic operations.

5.4 GPU Invoking Strategy

Obviously, invoking GPU for cooperating incurs extra costs. If the GPU invoking overhead is larger than the performance improvement we can obtain by exploiting CPU-GPU cooperative processing,

Table 2: The statistics of the used graph datasets

Graph	$ V $	$ E $	D_{avg}	Sink	Size	Domain
<i>gsh2015tpd</i> (GS)	31M	581M	18.8	24.9%	5GB	Web
<i>twitter2010</i> (TW)	42M	1,469M	35.2	4.5%	12GB	Social
<i>friendster</i> (FR)	124M	1,806M	14.56	18.1%	15GB	Social
<i>weibo</i> (WB)	70M	2,586M	37.1	14.6%	21GB	Social
<i>uk-2006</i> (UK)	78M	2,965M	38.01	11.3%	25GB	Web
<i>uk-union</i> (UN)	134M	5,955M	41.2	10.1%	50GB	Web

then we do not need to invoke it. As the profiling result shown in Figure 1(a), the number of active edges at the beginning or ending iterations is quite small. For example, the number of active edges in the first 2 iterations and the last 10 iterations are less than 1,500. Intuitively, invoking GPU in these iterations is not a good choice as it cannot fully exploit the high-parallelism of GPU but incurs extra overhead. In CGgraph, we propose a simple-yet-effective GPU invoking strategy to decide whether it invokes GPU or not at the beginning of each iteration. In particular, the GPU processor only is invoked if and only if the number of active edges in the iteration is larger than the given threshold τ and the percentage of the active edges in subgraph G' is larger than the given threshold θ . The threshold parameters τ and θ in CGgraph are tunable for different datasets and different graph processing algorithms.

Determining the default value of τ and θ . In CGgraph, we set the threshold τ as 2.5 million as we observed that the processing time of each iteration which has more than 2.5 million active edges are obvious large during the internal profiling experiments. For θ , i.e., the percentage of the active edges in subgraph G' , we vary its value from 15% to 75% with τ set to 2.5 million, and test the overall performance of CGgraph. We set θ to 35% by default as it works well in all cases. Please refer to Appendix B of the technical report [2] for the detailed analysis.

6 EVALUATION

We evaluate the superiority of our proposed CGgraph via extensive experiments in this section.

6.1 Experimental Settings

Graph dataset. Table 2 shows the statistics of 6 real-world graph datasets, which are used in the evaluation. The scale of the graphs ranges from 5GB to 50GB. By following the setting of [40], we remove duplicate edges and self-loops from all graph datasets before graph processing. All these graph processing systems can process both directed and undirected graphs, however, the undirected graphs could be converted to directed ones by replacing each undirected edge with a pair of directed edges. Thus, all edges are directed in all experiments. The percentage of sink nodes in these graphs is from 4.5% to 24.9%.

State-of-the-art systems. We compared CGgraph with 7 state-of-the-art graph processing systems in 3 categories, i.e., CPU-only, GPU-only, and CPU-GPU heterogeneous systems:

- Galois(V4.0) [36]. It is a CPU-only graph processing system on a single machine. It is a state-of-the-art CPU-only system, we use the source code provided by its github repository [3].

- Gemini [58]. It is a distributed CPU-only graph processing system. It has notable shared-machine performance. To provide a fair comparison, we remove its distributed design (e.g., messaging abstraction) from its source code [4].
- Gunrock(V2.0) [51]. It is a GPU-only graph processing system that achieves good performance via its novel frontier-based approach. The source code is available at [6].
- Groute [9]. It is a GPU-only graph processing system, it enjoys excellent performance via its GPU-based asynchronous execution and communication. Its github repository is at [5].
- Totem [20]. It is a CPU-GPU heterogeneous graph processing system. It statically partitions the graph into two subgraphs, one is used by the CPU and the other is loaded into GPU global memory and used by the GPU. The updated vertex values of each subgraph are synchronized after each iteration. The source code is available at [8].
- Subway [39]. It is also a CPU-GPU heterogeneous graph processing system. It generates a subgraph that contains only active edges and vertices on GPU memory. To further improve the performance, it reduces both the number of times the subgraph is loaded and the size of the loaded subgraph each time. We obtain its source code at [7].
- LargeGraph [55]. It is a CPU-GPU heterogeneous graph processing system, which uses a dependency-aware data-driven execution approach to achieve good performance. We cannot obtain the source code from the authors of [55], thus, we tried our best to re-implement it by ourselves.

Graph algorithms. We run 4 representative graph algorithms in this work: breadth-first search (BFS), single-source shortest path (SSSP), weakly connected component (WCC), and PageRank (PR). For BFS and SSSP, we use the same source vertex when comparing with other systems, which are randomly selected from the vertex set with the out-degree not equal to zero. Since SSSP requires graphs with edge weights, we randomly assign weight values between (0, 100] to every edge by following the setting of the existing systems [58]. For PR, we use the same terminal condition with 0.85 as the damping factor and run it for 20 iterations. The reported measurement is the average of processing the algorithm 10 times.

Implementation and hardware configurations. CGgraph is implemented by over 8000 lines of C++ and CUDA code, which is open-sourced at [1]. We compiled it with GCC 11.3 and CUDA 11.7 on Ubuntu 18.04 with O3-level optimization. We compare CGgraph with other 7 systems on a modern commodity machine with CPU-GPU co-processor by default. On the host, there is an Intel i9-9900k CPU with 8 cores and 128GB memory. On the device, there is an NVIDIA TITAN Xp GPU with 3840 CUDA cores and 12GB GDDR5 global memory. GPU connects with the host via the PCI Express 3.0 at 16x, which bandwidth is approximately 11 GB/s.

6.2 Overall Performance Evaluation

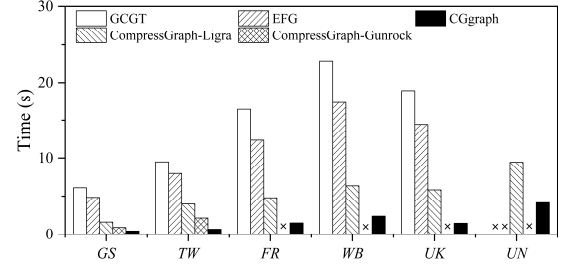
End-to-end processing time evaluation. Table 3 presents the end-to-end processing time (in seconds) of all evaluated graph systems when running 4 different algorithms on 6 graph datasets. Firstly, it is no doubt that CGgraph performs the best among all

Table 3: The execution time (in seconds) of compared systems

Alg	G	CPU-only		GPU-only		CPU-GPU heterogeneous system			
		Galois	Gemini	Gunrock	Groute	Totem	Subway	LargeGraph	CGgraph
BFS	GS	0.93	1.01	0.32	0.57	0.89	0.81	0.73	0.43
	TW	2.24	2.08	-	1.01	2.13	1.89	1.53	0.66
	FR	5.20	4.06	-	-	3.05	5.21	4.13	1.49
	WB	6.97	6.43	-	-	5.01	6.19	5.12	2.37
	UK	5.15	6.03	-	-	4.74	4.79	1.39	1.45
	UN	10.42	11.85	-	-	-	-	5.39	4.15
SSSP	GS	3.42	3.33	0.94	1.59	2.47	2.01	1.13	0.77
	TW	11.81	9.23	-	-	5.50	9.25	8.13	2.52
	FR	40.96	31.19	-	-	17.73	16.30	15.27	3.43
	WB	52.73	48.67	-	-	-	23.50	21.24	8.71
	UK	28.63	31.13	-	-	-	-	5.39	4.15
	UN	39.75	-	-	-	-	-	31.33	15.21
WCC	GS	2.97	3.73	0.81	0.77	1.81	1.73	1.34	0.79
	TW	6.41	8.14	-	3.20	5.97	5.73	5.35	1.59
	FR	26.35	22.21	-	-	8.14	10.28	8.76	2.38
	WB	37.57	32.13	-	-	14.73	16.32	14.11	5.01
	UK	15.13	29.42	-	-	17.81	20.37	11.92	7.54
	UN	25.72	35.95	-	-	-	-	18.43	12.71
PR	GS	13.72	12.57	3.07	4.56	5.42	4.07	5.49	2.46
	TW	58.14	44.83	-	16.12	25.46	23.13	20.50	5.97
	FR	92.70	75.37	-	-	41.23	33.73	31.46	11.83
	WB	118.59	98.27	-	-	48.27	44.73	45.82	18.68
	UK	57.82	38.51	-	-	23.49	20.57	19.27	8.30
	UN	69.21	54.61	-	-	-	-	44.26	24.75

compared systems. In particular, CGgraph ranks 1st among all 8 systems in 21 over 24 test cases, see bold values in the last column of Table 3. In addition, it ranks 2nd in the rest three test cases. However, the performance of CGgraph are comparable with the best in these three cases. Secondly, the GPU-only systems (i.e., Gunrock and Groute) fail to process large graphs as they encountered out-of-memory errors during graph processing, which are indicated with “-” in Table 3. Moreover, the CPU-GPU heterogeneous graph processing systems outperform the CPU-only systems due to the utilization of high parallelism on the GPU. Thirdly, there is no clear winner among the existing CPU-GPU heterogeneous systems (i.e., Totem, Subway, and LargeGraph). Moreover, Totem and Subway cannot process the largest dataset UN in all tested algorithms as they encounter out-of-memory errors. Nevertheless, CGgraph outperforms all of them in almost all tested cases. The core reason is CGgraph addresses GPU memory over-subscription via extracting a subgraph that only load once but can be used by multiple iterations, and it also devises efficient CPU-GPU cooperative processing scheme, which are equipped optimization techniques for task processing on CPU and GPU. Fourthly, from the graph algorithm-wise, the speedup times of CGgraph over all other systems in SSSP (up to 11.94X) and PR (up to 9.73X) are significantly larger than its in BFS (up to 4.15X). It is because of the algorithmic-logic of SSSP and PR are more complex than BFS, which also indicates the superiority of CGgraph to process complex algorithms on large-scale graphs. Lastly, CGgraph is obviously better than other systems when processing social network graphs (e.g., TW, FR and WB) as the number of computation tasks in these graphs are extremely large in some iterations and the efficient cooperative processing scheme in CGgraph could balance the workload in both vertex-level and edge-level on CPU and GPU.

Comparison with compress-based systems. In the literature, several studies have been proposed to run graph algorithms in compressed graphs. We compare the performance of running BFS on three compress-based graph systems (i.e., GCGT [42], EFG [18] and CompressGraph [12]) and our CGgraph in 6 graph datasets. In particular, we adapt both Ligra [44] and Gunrock [51] to run

**Figure 12: The performance of running BFS on compress-based graph systems and CGgraph**

BFS on CompressGraph as it is a generic compression framework. The results are plotted in Figure 12, where indicates the system cannot return result due to runtime error. Obviously, CGgraph performs the best. In particular, the speedup times of CGgraph over these compress-based competitors ranges from 1.81X to 14.37X. Interestingly, Gunrock fails to run BFS on TW (see Table 3), but it takes 2.12 seconds with the compressed TW via CompressGraph.

6.3 Effectiveness of Designed Techniques

6.3.1 Effectiveness of Graph Reordering. In this section, we evaluate the effectiveness of the proposed graph reordering algorithm (see Section 4.2) in CGgraph. In particular, we first verify the effectiveness of different graph reordering algorithms on the end-to-end performance of CGgraph, then we demonstrate the performance improved by adapting our graph reordering algorithm to other graph processing systems.

Comparison among different graph reordering algorithms. In this experiment, we compare the following 6 graph reordering algorithms.

- Default, it uses the default vertices order in the input graph and did not apply any graph reordering algorithm on it in CGgraph.
- Random, we follow other graph systems [19, 52] to randomly shuffle the vertices in the input graph and use the shuffled vertex array in CGgraph.
- In-degree [17], it sorts the vertices in the input graph by the descending order of their in-degree.
- BP [15], the vertices in the input graph are reordered in a compression-friendly manner via recursive graph bisection.
- RCM [14], it employs a BFS-based algorithm to permute a sparse adjacency matrix by reducing its bandwidth.
- Gorder [52], it minimizes cache misses by maximizing the shared edges within consecutive node blocks of size w .
- CGgraph, it is our proposal, which utilizes the proposed graph reordering algorithm in Section 4.2.

Figure 13 depicts the speedup times of different graph reordering algorithms over the default vertex order in CGgraph. We report SSSP and PR on two different datasets FR and UK and omit others due to page limits. The speedup times of CGgraph over the Default of SSSP and PR on FR are 1.72 and 1.57, and on UK are 1.52 and 1.49, respectively. It consistently outperforms all alternative options because our graph reordering algorithm in CGgraph takes both the in-degree of vertices and the data access locality into consideration.

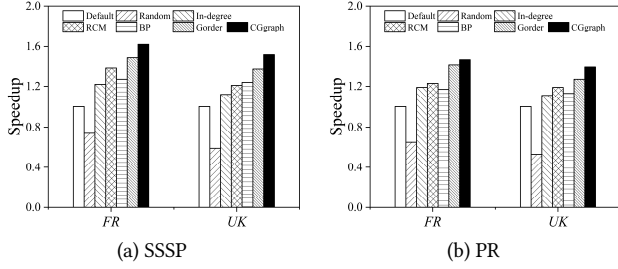


Figure 13: Comparison of different graph reordering methods

Table 4: The speedup times via CGraph’s graph reordering

System		Galois	Gemini	Totem	Subway	LargeGraph
SSSP	FR	3.01X	2.85X	2.37X	2.26X	2.31X
	UK	3.48X	3.01X	-	2.38X	1.49X
PR	FR	4.66X	3.86X	2.45X	2.24X	2.07X
	UK	4.01X	3.13X	2.19X	2.01X	1.82X

In addition, the Random order performs even worse than Default as it does not exhibit the data access locality in the original graph.

Performance improvement of other systems. In this experiment, we evaluate the generality of our proposed graph reordering algorithm by using the reordered graph in existing 5 graph processing systems, i.e., Galois, Gemini, Totem, Subway and LargeGraph. Gunrock and Groute are ignored as they cannot process large graphs as FR and UK. More specifically, we only change the input graph of these systems and do not revise any execution logic in these existing systems.

Since Totem fails to process SSSP on the original UK (see the corresponding “-” in Table 3), it is not surprising it also fails with the reordered UK (in Table 4). Except it, our graph reordering algorithm improves the performance of all these 5 systems in all tested cases, as shown in Table 4. The overall speedup times among these systems range from 1.82X to 4.66X. Interestingly, even in the recently proposed CPU-GPU heterogeneous graph processing systems (i.e., Subway and LargeGraph), our reordering algorithm brings up to 2.38X speedup in the tested cases. The core reason is our graph reordering algorithms offer excellent data access locality, which significantly reduces cache misses during graph processing.

6.3.2 Effectiveness of Subgraph Extraction Approach. We then evaluate the effectiveness of the subgraph extraction approach in this section. In particular, there are three approaches for extracting a subgraph from the reordered graph. The first approach is randomly extracting a subset of vertices and their corresponding edges to form the subgraph, we use it as the baseline method. The second approach is extracting the vertices and all of their neighbors as much as possible from the reordered vertices list, we refer to it as *vertex-based* method. The third approach is using our size-constrained subgraph extraction approach in Section 4.3, we refer it as *vertex-edge-based* method.

Table 5 reports the speedup times of *vertex-based* and *vertex-edge-based* approaches over the baseline method when processing SSSP and PR on FR and UK. Specifically, both end-to-end processing time and vertex value synchronization cost are reported. The synchronization cost is updating the vertex values between GPU memory and CPU memory (see Figure 8), which includes both the PCIe cost and the value updating cost in the CPU. First, both

Table 5: The speedup times of subgraph extraction approach

Algorithm	Graph	End-to-end		Synchronization	
		<i>vertex</i>	<i>vertex-edge</i>	<i>vertex</i>	<i>vertex-edge</i>
SSSP	FR	1.51X	1.67X	1.01X	1.21X
	UK	1.50X	1.74X	1.00X	1.69X
PR	FR	1.33X	1.45X	1.00X	1.02X
	UK	1.30X	1.46X	1.00X	1.14X

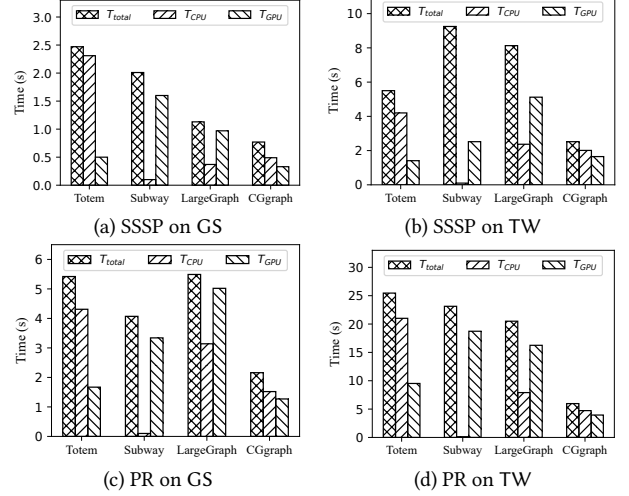


Figure 14: Breakdown of graph processing time

vertex-based and *vertex-edge-based* approaches are better than the baseline approach, it confirms the effectiveness of our idea to extract a subgraph that can be used in multiple iterations. Second, the *vertex-edge-based* approach achieves 1.02X to 1.69X improvements on synchronization cost, which is higher than the *vertex-based* approach. The reason is that the number of synchronized values in *vertex-edge-based* approach is smaller than the *vertex-based* approach as only a subset of edges of every selected vertex will be included in the extracted subgraph in our approach.

6.3.3 Effectiveness of CPU-GPU Cooperative Processing. We next evaluate the effectiveness of our CPU-GPU cooperative processing scheme in CGraph. We run SSSP and PR on all these 4 CPU-GPU heterogeneous systems. In this experiment, we use the two smallest datasets, i.e., GS and TW, as existing CPU-GPU systems fail to process large graphs in some tested cases. We measure the end-to-end processing time (T_{total}), CPU processing time (T_{CPU}) and GPU processing time (T_{GPU}), and plot them in Figure 14.

First of all, the end-to-end processing time of CGraph is the smallest among all 4 systems in every tested case, as T_{total} bar shown in Figure 14. Secondly, the gap between the CPU processing time (T_{CPU}) and the GPU processing time (T_{GPU}) of CGraph is significantly smaller than the other systems as we explicitly consider the workload balance between CPU and GPU via on-demand task allocation in Section 5.1. Thirdly, the end-to-end processing time (T_{total}) of Subway and LargeGraph in all tested cases are obviously larger than their CPU processing time (T_{CPU}) as both systems do not balance the workload on CPU and GPU, and the data movement or the subgraph generation in them incurs extra overhead.

6.3.4 Effectiveness of Optimizations on CPU. In this section, we verify the effectiveness of our proposed optimization techniques on the CPU. The baseline idea to process the tasks on the CPU is equally

Table 6: Comparison of workload balance methods on CPU

Algorithm	Graph	V-Donating	V-Stealing	V/E-Stealing
SSSP	FR	1.11X	1.33X	1.41X
	UK	1.17X	1.21X	1.33X
PR	FR	1.12X	1.30X	1.36X
	UK	1.10X	1.35X	1.44X

Table 7: Comparison between CTA and IWB

Algorithm	Graph	L1 Hit ratio		AT/warp		Speedup
		CTA	IWB	CTA	IWB	
SSSP	GS	41.2%	65.7%	19.14	22.86	1.30X
	TW	33.3%	56.8%	21.18	22.59	1.44X
PR	GS	45.9%	70.7%	19.41	23.63	1.33X
	TW	44.9%	66.6%	24.20	25.98	1.38X

dividing them into the task queue of every core on the CPU, then each core exclusively executes the assigned tasks in its task queue. To improve the load balance among different cores in the CPU, there are three methods have been explored: (i) V-Donating [25]. An overloaded core shares its tasks with underloaded cores at the vertex-level; (ii) V-Stealing [4]. An underloaded core steals tasks from overloaded cores at the vertex-level; and (iii) V/E-Stealing. Our proposed solution in Section 5.2, which allows the underloaded core to steal tasks from overloaded cores at both vertex-level and edge-level.

Table 6 shows the speedup times of different methods over the baseline by running SSSP and PR on FR and UK. It is no doubt all the above three CPU load balance solutions achieve better performance when compared with the baseline method, which does not balance the workload among different cores on the CPU at all. Even both V-Stealing and V-Donating methods balance the workload at vertex-level, V-Stealing is slightly better than V-Donating as V-Donating incurs extra cost to collect the status of other cores and write shared tasks into shared memory. Last but the most important, our proposed V/E-Stealing performs the best among all these methods. The key reason is that it guarantees the fine-grained workload balance of different cores on the CPU.

6.3.5 Effectiveness of Optimizations on GPU. We last evaluate the impact of our Inter-Warp balancing (IWB) optimization for task processing on GPU. In particular, we compare the performance of IWB with the widely-used CTA by running SSSP and PR on GS and TW. We measure and report the following three metrics [47, 49]: (i) L1 cache hit ratio, (ii) the average active threads per warp (denoted as AT/Warp), and (iii) the end-to-end speedup times of CGgraph with IWB over CGgraph with CTA.

As depicted in Table 7, IWB not only achieves high GPU utilization (see AT/WARP) but also increases the L1 cache hit ratio. Since IWB allows the idle threads to fetch tasks to run from the shared memory, the average activated threads in the warp of IWB is larger than it is of CTA. The L1 cache hit ratio of IWB is higher than that of CTA, e.g., it ranges from 56.8% to 70.7% in all tested cases with IWB, but it is only 33.3% to 45.9% with CTA. This is because IWB explicitly moves the medium-degree vertices to L1 cache. The high L1 cache hit ratio and more running threads per warp lead to the speedup times of IWB over CTA ranging from 1.33X to 1.44X.

Table 8: Performance evaluation on a high-end server

G	SSSP			WCC			PR		
	HS-BoE	SM-CGg	HS-CGg	HS-BoE	SM-CGg	HS-CGg	HS-BoE	SM-CGg	HS-CGg
GS	0.40	0.77	0.55	0.45	0.79	0.52	1.59	2.46	1.73
TW	2.10	2.52	1.57	1.11	1.59	1.05	4.27	5.97	3.65
FR	10.27	3.43	2.07	4.73	2.38	1.45	18.63	11.83	9.43
WB	14.49	8.71	4.79	5.26	5.01	3.11	27.75	18.68	14.13
UK	7.54	4.15	3.89	4.51	7.54	5.31	13.76	8.30	5.46
UN	21.18	15.21	9.27	13.07	12.71	8.99	28.39	24.75	15.45

6.4 Performance Evaluation on High-end Server

We evaluate CGgraph and other systems on a high-end CPU-GPU server, which was equipped with a 12-core Intel Xeon Silver 4214 CPU and an NVIDIA Tesla V100S GPU featuring 5120 cores and 32GB of global memory. Specifically, we compare the best performance of existing systems on the high-end server (i.e., HS-BoE), the performance of CGgraph on a single modern commodity machine (i.e., SM-CGg), and the performance of CGgraph on the high-end server (i.e., HS-CGg). In Table 8, we show the experimental results by running SSSP, WCC and PR in these 6 datasets, and refer the interested readers to our technical report [2] for complete experimental results. The key findings are two-fold: (i) CGgraph consistently outperforms existing systems with the high-end server, see the values in the columns of HS-CGg in Table 8; (ii) the performance of our CGgraph on modern commodity CPU-GPU co-processor is better than (see bold values in Table 8) or at least comparable with the best performance of other systems on high-end servers. For example, the best performance of existing systems to process SSSP on UN with a high-end server is achieved by LargeGraph, which takes 21.18s. However, CGgraph only takes 15.21s to process SSSP on UN on a modern commodity machine.

7 CONCLUSION

In this paper, we propose an ultra-fast graph processing system CGgraph on a modern commodity machine with the CPU-GPU co-processor. CGgraph achieves significant speedup by overcoming GPU memory over-subscription and providing an efficient CPU-GPU cooperative processing scheme. In the evaluation, we compare CGgraph with 7 state-of-the-art graph processing systems in various experimental settings. The results indicate that, CGgraph achieves a substantial performance improvement on a commodity CPU-GPU co-processor. In the future, we plan to study the distributed CPU-GPU heterogeneous graph processing system and consider hardware-software co-design for graph algorithms with CPU-GPU co-processor.

ACKNOWLEDGMENTS

We thank all reviewers for constructive comments. Dr. Ye Yuan is supported by the National Key RD Program of China (Grant No. 2022YFB2702100), the NSFC (Grant Nos. 61932004, 62225203, U21A20516) and the DITDP (Grant No. JCKY2021211B017). Dr. Bo Tang was partially supported by Shenzhen Fundamental Research Program (Grant No. 20220815112848002), the Guangdong Provincial Key Laboratory (Grant No. 2020B121201001) and a research gift from Huawei Gauss department. He is also affiliated with the Research Institute of Trustworthy Autonomous Systems, Southern University of Science and Technology, Shenzhen, China.

REFERENCES

- [1] 2023. *CGgraph source code*. <https://github.com/DBGGroup-SUSTech/CGgraph>
- [2] 2023. *CGgraph (Technical Report)*. <https://github.com/PengBo410/CGgraphV1/blob/master/src/TR/TR.pdf>
- [3] 2023. *Galois*. <https://github.com/IntelligentSoftwareSystems/Galois>
- [4] 2023. *Gemini*. <https://github.com/thu-pacman/GeminiGraph>
- [5] 2023. *Groute*. <https://github.com/groute/groute>
- [6] 2023. *Gunrock*. <https://github.com/gunrock/gunrock>
- [7] 2023. *Subway*. <https://github.com/AutomataLab/Subway>
- [8] 2023. *Totem*. <https://github.com/netsyslab/Totem>
- [9] Tal Ben-Nun, Michael Sutton, Sreepathi Pai, and Keshav Pingali. 2017. Groute: An asynchronous multi-GPU programming model for irregular computations. *ACM SIGPLAN Notices* 52, 8 (2017), 235–248.
- [10] Paolo Boldi, Marco Rosa, Massimo Santini, and Sebastiano Vigna. 2011. Layered label propagation: A multiresolution coordinate-free ordering for compressing social networks. In *WWW*. 587–596.
- [11] Jie Chen and Yousef Saad. 2010. Dense subgraph extraction with application to community detection. *TKDE* 24, 7 (2010), 1216–1230.
- [12] Zheng Chen, Feng Zhang, JiaWei Guan, Jidong Zhai, Xipeng Shen, Huanchen Zhang, Wentong Shu, and Xiaoyong Du. 2023. CompressGraph: Efficient Parallel Graph Analytics with Rule-Based Compression. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–31.
- [13] Michael Curtiss, Iain Becker, Tudor Bosman, Sergey Doroshenko, Lucian Grijincu, Tom Jackson, Sandhya Kunnatur, Soren Lassen, Philip Pronin, and Sriram Sankar. 2013. Unicorn: A system for searching the social graph. *PVLDB* 6, 11 (2013), 1150–1161.
- [14] Elizabeth Cuthill and James McKee. 1969. Reducing the bandwidth of sparse symmetric matrices. In *ACM National Conference*. 157–172.
- [15] Laxman Dhulipala, Igor Kabiljo, Brian Karrer, Giuseppe Ottaviano, Sergey Pupyrev, and Alon Shalita. 2016. Compressing graphs and indexes with recursive graph bisection. In *SIGKDD*. 1535–1544.
- [16] James Dinan, D Brian Larkins, Ponnuswamy Sadayappan, Sriram Krishnamoorthy, and Jarek Nieplocha. 2009. Scalable work stealing. In *SC*. 1–11.
- [17] Priyank Faldut, Jeff Diamond, and Boris Grot. 2019. A closer look at lightweight graph reordering. In *ISWC*. IEEE, 1–13.
- [18] Prasun Gera and Hyesoon Kim. 2023. Traversing Large Compressed Graphs on GPUs. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 25–35.
- [19] Prasun Gera, Hyeojong Kim, Piyush Sao, Hyesoon Kim, and David Bader. 2020. Traversing large graphs on GPUs with unified memory. *PVLDB* 13, 7 (2020), 1119–1133.
- [20] Abdullah Gharaibeh, Lauro Beltrão Costa, Elizeu Santos-Neto, and Matei Ripeanu. 2012. A yoke of oxen and a thousand chickens for heavy lifting graph processing. In *PACT*. 345–354.
- [21] Joseph E Gonzalez, Yucheng Low, Haijie Gu, Danny Bickson, and Carlos Guestrin. 2012. Powergraph: Distributed graph-parallel computation on natural graphs. In *OSDI*. 17–30.
- [22] Joseph E Gonzalez, Reynold S Xin, Ankur Dave, Daniel Crankshaw, Michael J Franklin, and Ion Stoica. 2014. Graphx: Graph processing in a distributed dataflow framework. In *OSDI*. 599–613.
- [23] Wei Han, Daniel Mawhirther, Bo Wu, and Matthew Buland. 2017. Graphie: Large-scale asynchronous graph traversals on just a GPU. In *PACT*. 233–245.
- [24] Hideya Iwasaki, Kento Emoto, Akimasa Morihata, Kiminori Matsuzaki, and Zhenjiang Hu. 2022. Fregel: a functional domain-specific language for vertex-centric large-scale graph processing. *Journal of Functional Programming* 32 (2022), e4.
- [25] Joseph John, Josh Milthorpe, and Peter Strazdins. 2022. Distributed Work Stealing in a Task-Based Dataflow Runtime. In *PPAM*. Springer, 225–236.
- [26] Farzad Khorasani, Keval Vora, Rajiv Gupta, and Laxmi N Bhuyan. 2014. CuSha: vertex-centric graph processing on GPUs. In *HPDC*. 239–252.
- [27] Min-Soo Kim, Kyuhyeon An, Himchan Park, Hyunseok Seo, and Jinwook Kim. 2016. GTS: A fast and scalable graph processing method based on streaming topology to GPUs. In *SIGMOD*. 447–461.
- [28] Kartik Lakhotia, Rajgopal Kannan, Sourav Pati, and Viktor Prasanna. 2020. Gpop: A scalable cache-and memory-efficient framework for graph processing over parts. *TOPC* 7, 1 (2020), 1–24.
- [29] Victor E Lee, Ning Ruan, Ruoming Jin, and Charu Aggarwal. 2010. A survey of algorithms for dense subgraph discovery. *Managing and mining graph data* (2010), 303–336.
- [30] Chen Li, Rachata Ausavarungrun, Christopher J Rossbach, Youtao Zhang, Onur Mutlu, Yang Guo, and Jun Yang. 2019. A framework for memory oversubscription management in graphics processing units. In *ASPLOS*. 49–63.
- [31] Hang Liu and H Howie Huang. 2015. Enterprise: breadth-first graph traversal on GPUs. In *SC*. 1–12.
- [32] Haotian Liu, Bo Tang, Jiashu Zhang, Yangshen Deng, Xiao Yan, Xinying Zheng, Qiaomu Shen, Dan Zeng, Zunyao Mao, Chaozu Zhang, et al. 2022. GHive: accelerating analytical query processing in apache hive via CPU-GPU heterogeneous computing. In *SoCC*. 158–172.
- [33] Grzegorz Malewicz, Matthew H Austern, Aart JC Bik, James C Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. 2010. Pregel: a system for large-scale graph processing. In *SIGMOD*. 135–146.
- [34] Ke Meng, Jiajia Li, Guangming Tan, and Ninghui Sun. 2019. A pattern based algorithmic autotuner for graph processing on GPUs. In *PPoPP*. 201–213.
- [35] Duane Merrill, Michael Garland, and Andrew Grimshaw. 2012. Scalable GPU graph traversal. *ACM SIGPLAN Notices* 47, 8 (2012), 117–128.
- [36] Donald Nguyen, Andrew Lenharth, and Keshav Pingali. 2013. A lightweight infrastructure for graph analytics. In *SOSP*. 456–471.
- [37] Xiafei Qiu, Wubin Cen, Zhengping Qian, You Peng, Ying Zhang, Xuemin Lin, and Jingren Zhou. 2018. Real-time constrained cycle detection in large dynamic graphs. *PVLDB* 11, 12 (2018), 1876–1888.
- [38] Abdul Qamar, Amol Deshpande, and Jimmy Lin. 2016. Nscale: neighborhood-centric large-scale graph analytics in the cloud. *The VLDB Journal* 25 (2016), 125–150.
- [39] Amir Hossein Nodehi Sabet, Zhijia Zhao, and Rajiv Gupta. 2020. Subway: Minimizing data transfer during out-of-GPU-memory graph processing. In *EuroSys*. 1–16.
- [40] Peter Sanders, Christian Schulz, and Dorothea Wagner. 2014. Benchmarking for graph clustering and partitioning. *ESNAM* (2014).
- [41] Dipanjan Sengupta, Shuaiwen Leon Song, Kapil Agarwal, and Karsten Schwan. 2015. Graphreduce: processing large-scale graphs on accelerator-based systems. In *SC*. 1–12.
- [42] Mo Sha, Yuchen Li, and Kian-Lee Tan. 2019. Gpu-based graph traversal on compressed graphs. In *Proceedings of the 2019 International Conference on Management of Data*. 775–792.
- [43] Xuanhua Shi, Xuan Luo, Junling Liang, Peng Zhao, Sheng Di, Bingsheng He, and Hai Jin. 2017. Frog: Asynchronous graph processing on GPU with hybrid coloring model. *TKDE* 30, 1 (2017), 29–42.
- [44] Julian Shun and Guy E Blelloch. 2013. Ligma: a lightweight graph processing framework for shared memory. In *PPoPP*. 135–146.
- [45] Yogesh Simmhan, Alok Kumbhare, Charith Wickramaarachchi, Soonil Nagarkar, Santosh Ravi, Cauligi Raghavendra, and Viktor Prasanna. 2014. Goffish: A sub-graph centric framework for large-scale graph analytics. In *Euro-Par 2014 Parallel Processing: 20th International Conference, Porto, Portugal, August 25-29, 2014. Proceedings* 20. Springer, 451–462.
- [46] Yuanyuan Tian, Andrey Balmin, Severin Andreas Corsten, Shirish Tatikonda, and John McPherson. 2013. From "think like a vertex" to "think like a graph". *Proceedings of the VLDB Endowment* 7, 3 (2013), 193–204.
- [47] Charbel Toumeh and Alain Lambert. 2021. Gpu accelerated voxel grid generation for fast mav exploration. *arXiv:2112.13169* (2021).
- [48] Hao Wang, Liang Geng, Rubao Lee, Kaixi Hou, Yanfeng Zhang, and Xiaodong Zhang. 2019. SEP-graph: finding shortest execution paths for graph processing under a hybrid framework on GPU. In *PPoPP*. 38–52.
- [49] Pengyu Wang, Jing Wang, Chao Li, Jianzong Wang, Haojin Zhu, and Minyi Guo. 2021. Grus: Toward unified-memory-efficient high-performance graph processing on gpu. *TACO* 18, 2 (2021), 1–25.
- [50] Qiang Wang, Xin Ai, Yanfeng Zhang, Jing Chen, and Ge Yu. 2023. HyTGraph: GPU-Accelerated Graph Processing with Hybrid Transfer Management. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 558–571.
- [51] Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D Owens. 2016. Gunrock: A high-performance graph processing library on the GPU. In *PPoPP*. 1–12.
- [52] Hao Wei, Jeffrey Xu Yu, Can Lu, and Xuemin Lin. 2016. Speedup graph processing by graph ordering. In *SIGMOD*. 1813–1828.
- [53] Ye Yuan, Guoren Wang, Lei Chen, and Haixun Wang. 2013. Efficient keyword search on uncertain graph data. *TKDE* 25, 12 (2013), 2767–2779.
- [54] Yunming Zhang, Vladimir Kiriansky, Charith Mendis, Saman Amarasinghe, and Matei Zaharia. 2017. Making caches work for graph analytics. In *Big Data*. IEEE, 293–302.
- [55] Yu Zhang, Da Peng, Xiaofei Liao, Hai Jin, Haikun Liu, Lin Gu, and Bingsheng He. 2021. LargeGraph: An efficient dependency-aware GPU-accelerated large-scale graph processing. *TACO* 18, 4 (2021), 1–24.
- [56] Yunming Zhang, Mengjiao Yang, Riyadh Baghdadi, Shoaib Kamil, Julian Shun, and Saman Amarasinghe. 2018. Graphit: A high-performance graph dsl. *PACMPL* 2, OOPSLA (2018), 1–30.
- [57] Jianlong Zhong and Bingsheng He. 2013. Medusa: Simplified graph processing on GPUs. *TPDS* 25, 6 (2013), 1543–1552.
- [58] Xiaowei Zhu, Wenguang Chen, Weimin Zheng, and Xiaosong Ma. 2016. Gemini: A computation-centric distributed graph processing system.. In *OSDI*, Vol. 16. 301–316.